
Guía de Ejercicios N°4 – Ciclos en C

Ayudante: Matías Fuentes

1. Detector de Drones

Un sistema de vigilancia recibe señales enteras que representan lecturas captadas desde el cielo. Cada señal puede ser **1** (se detectó movimiento) o **0** (no se detectó movimiento). El sistema debe leer señales de forma continua hasta que se ingrese el valor **-1**, el cual indica el término de la transmisión y no debe considerarse una señal válida.

Cada vez que el sistema reciba una nueva señal, debe evaluar las **últimas 3 señales captadas** (la actual y las dos anteriores): si las tres señales consecutivas son iguales a 1, el sistema debe imprimir "¡Alerta! posible dron detectado". Si aún no se han recibido al menos 3 señales, no se debe realizar ninguna evaluación.

Hint: Puedes resolverlo utilizando tres variables enteras que vayan “desplazando” los valores recibidos (por ejemplo, antes de leer la nueva señal, actualiza: la más antigua toma el valor de la intermedia, la intermedia toma el valor de la más reciente, y luego lees el nuevo valor). Recuerda que **no debes utilizar** `while(1)` para tu ciclo principal: apóyate en la condición de lectura dentro del `while` o `do-while`.

2. Número Palíndromo

Escriba un programa en C que solicite números enteros positivos de un solo dígito (entre 0 y 9) al usuario, uno a la vez, e ir concatenándolos para formar un único número. El programa debe seguir solicitando dígitos hasta que el usuario ingrese un valor **fuera del rango** 0-9, momento en el cual se detiene la lectura (ese último valor no se concatena).

Una vez formado el número completo, el programa debe:

- Calcular su número **invertido** (el número "dado vuelta").
- Indicar si el número original es **palíndromo** (es decir, si es igual a su versión invertida).
- Indicar cuál de los dos números es **mayor**, en caso de que no sean iguales.

Hint: Para ir concatenando dígitos puedes usar la operación `numero = numero * 10 + digito`. Para invertir un número entero, utiliza un ciclo que vaya extrayendo el último dígito con el operador módulo (`% 10`) y lo vaya agregando a un nuevo número, mientras reduces el original con división entera (`/ 10`).

3. Contador de Aterrizajes

Una pista de aterrizaje recibe señales enteras que representan la velocidad (en km/h) de las naves que se acercan. El sistema debe leer velocidades de forma continua hasta que se ingrese un valor **negativo**, el cual indica el cierre de la pista y no debe considerarse una lectura válida.

El programa debe contar cuántas naves aterrizaron a **velocidad segura** (menor o igual a 120 km/h) y cuántas lo hicieron a **velocidad de riesgo** (mayor a 120 km/h). Al finalizar la lectura, se debe imprimir ambos contadores, además del **promedio** de todas las velocidades registradas.

Hint: Utiliza dos variables contador (una para cada categoría) y una variable acumuladora para la suma de velocidades, junto con otra que cuente el total de naves para calcular el promedio al final. No olvides evitar la división por cero si no se ingresó ninguna velocidad válida.

4. Suma de Dígitos y Validación

Escriba un programa en C que solicite al usuario un número entero positivo y calcule la **suma de sus dígitos**. Por ejemplo, si se ingresa 4831, la suma de sus dígitos es $4 + 8 + 3 + 1 = 16$.

Además, el programa debe determinar si dicha suma es un número **par** o **impar**, e imprimir el resultado correspondiente.

Ejercicio extra: Implemente esta lógica dentro de una **función** `int sumarDigitos(int numero)` que reciba el número y retorne la suma de sus dígitos, la cual debe ser invocada desde la función `main`.

Hint: Al igual que en el ejercicio de invertir un número, puedes usar el operador módulo (`% 10`) dentro de un ciclo `while` para ir extrayendo cada dígito uno a uno, sumándolo a un acumulador, mientras reduces el número original con división entera (`/ 10`) hasta que este llegue a 0.

5. Suma de los Primeros N Términos de Fibonacci

La secuencia de Fibonacci se define como: $F(0) = 0$, $F(1) = 1$, y $F(n) = F(n - 1) + F(n - 2)$ para $n \geq 2$. Escriba un programa en C que solicite al usuario un valor entero N (mayor o igual a 1) y calcule la **suma de los primeros N términos** de la secuencia de Fibonacci (comenzando desde $F(0)$).

Ejercicio extra: Implemente una **función recursiva** `int fibonacci(int n)` que calcule y retorne el n -ésimo término de la secuencia, y utilícela desde el `main` dentro de un ciclo para ir sumando cada uno de los N primeros términos.

Hint: La función recursiva debe considerar dos **casos base**: si $n = 0$ retorna 0, y si $n = 1$ retorna 1. Para cualquier otro caso, debe retornar la suma de `fibonacci(n-1)` y `fibonacci(n-2)`. Ten en cuenta que esta solución, aunque simple de escribir, es poco eficiente para valores grandes de N , ya que recalcula muchos términos repetidamente.

Soluciones

Solución Ejercicio 1: Detector de Drones

```
1 #include <stdio.h>
2
3 int main() {
4     int senal;
5     int anterior2 = 0, anterior1 = 0, actual = 0;
6
7     printf("--- SISTEMA DE VIGILANCIA ---\n");
8     printf("Ingrese senales (1 o 0). Ingrese -1 para terminar: ");
9     scanf("%d", &senal);
10
11     while (senal != -1) {
12         // Desplazamos los valores: el mas antiguo se descarta
13         anterior2 = anterior1;
14         anterior1 = actual;
15         actual = senal;
16
17         // Evaluamos las 3 ultimas senales
18         if (anterior2 == 1 && anterior1 == 1 && actual == 1) {
19             printf("Alerta: posible dron detectado\n");
20         }
21
22         printf("Ingrese la siguiente senal (-1 para terminar): ");
23         scanf("%d", &senal);
24     }
25
26     printf("Transmision finalizada.\n");
27
28     return 0;
29 }
```

Explicación: Usamos tres variables que se van "desplazando" en cada iteración para mantener siempre las últimas tres señales recibidas. Como anterior2 y anterior1 parten en 0, la condición no puede cumplirse falsamente antes de recibir al menos 3 señales reales de 1.

Solución Ejercicio 2: Número Palíndromo

```
1 #include <stdio.h>
2
3 int main() {
4     int digito;
5     int numero = 0;
6     int original, invertido = 0, resto;
7
8     printf("Ingrese digitos (0-9) uno a uno.\n");
9     printf("Ingrese un valor fuera de ese rango para detener: ");
10    scanf("%d", &digito);
11
12    while (digito >= 0 && digito <= 9) {
13        numero = numero * 10 + digito;
14        printf("Ingrese el siguiente digito: ");
15        scanf("%d", &digito);
16    }
17
18    original = numero;
19
20    // Invertimos el numero formado
21    while (numero > 0) {
22        resto = numero % 10;
23        invertido = invertido * 10 + resto;
24        numero = numero / 10;
25    }
26
27    printf("Numero formado: %d\n", original);
28    printf("Numero invertido: %d\n", invertido);
29
30    if (original == invertido) {
31        printf("El numero SI es palindromo.\n");
32    } else {
33        printf("El numero NO es palindromo.\n");
34        if (original > invertido) {
35            printf("El numero original es mayor.\n");
36        } else {
37            printf("El numero invertido es mayor.\n");
38        }
39    }
40
41    return 0;
42 }
```

Explicación: El ciclo de lectura concatena dígitos válidos mientras estén en el rango 0-9. Luego, un segundo ciclo `while` independiente construye el número invertido extrayendo dígitos con `%` y reduciendo el número con `/`, técnica base para muchos problemas de dígitos.

Solución Ejercicio 3: Contador de Aterrizajes

```
1 #include <stdio.h>
2
3 int main() {
4     int velocidad;
5     int contadorSeguras = 0, contadorRiesgo = 0, totalNaves = 0;
6     float sumaVelocidades = 0, promedio;
7
8     printf("Ingrese velocidades (km/h). Ingrese un valor negativo para terminar
9           : ");
10    scanf("%d", &velocidad);
11
12    while (velocidad >= 0) {
13        if (velocidad <= 120) {
14            contadorSeguras++;
15        } else {
16            contadorRiesgo++;
17        }
18
19        sumaVelocidades = sumaVelocidades + velocidad;
20        totalNaves++;
21
22        printf("Ingrese la siguiente velocidad (negativo para terminar): ");
23        scanf("%d", &velocidad);
24    }
25
26    printf("Aterrizajes seguros: %d\n", contadorSeguras);
27    printf("Aterrizajes de riesgo: %d\n", contadorRiesgo);
28
29    if (totalNaves > 0) {
30        promedio = sumaVelocidades / totalNaves;
31        printf("Velocidad promedio: %.2f km/h\n", promedio);
32    } else {
33        printf("No se registraron velocidades.\n");
34    }
35
36    return 0;
37 }
```

Explicación: Se mantienen dos contadores independientes para clasificar cada lectura y un acumulador junto a un contador total para el promedio. La validación `if (totalNaves > 0)` evita una división por cero si el usuario termina la lectura sin ingresar datos.

Solución Ejercicio 4: Suma de Dígitos y Validación

```
1 #include <stdio.h>
2
3 // Funcion que recibe un numero y retorna la suma de sus digitos
4 int sumarDigitos(int numero) {
5     int suma = 0;
6     int resto;
7
8     while (numero > 0) {
9         resto = numero % 10;
10        suma = suma + resto;
11        numero = numero / 10;
12    }
13
14    return suma;
15 }
16
17 int main() {
18     int numero, suma;
19
20     printf("Ingrese un numero entero positivo: ");
21     scanf("%d", &numero);
22
23     suma = sumarDigitos(numero);
24
25     printf("La suma de los digitos es: %d\n", suma);
26
27     if (suma % 2 == 0) {
28         printf("La suma es un numero PAR.\n");
29     } else {
30         printf("La suma es un numero IMPAR.\n");
31     }
32
33     return 0;
34 }
```

Explicación: La función `sumarDigitos` encapsula la lógica de extracción de dígitos usando `%` y `/`, permitiendo que `main` se enfoque solo en pedir el dato y evaluar la paridad del resultado con el operador módulo.

Solución Ejercicio 5: Suma de los Primeros N Términos de Fibonacci

```
1 #include <stdio.h>
2
3 // Funcion recursiva que calcula el n-esimo termino de Fibonacci
4 int fibonacci(int n) {
5     if (n == 0) {
6         return 0;
7     } else if (n == 1) {
8         return 1;
9     } else {
10        return fibonacci(n - 1) + fibonacci(n - 2);
11    }
12 }
13
14 int main() {
15     int N, i;
16     int sumaTotal = 0;
17
18     printf("Ingrese la cantidad N de terminos a sumar: ");
19     scanf("%d", &N);
20
21     for (i = 0; i < N; i++) {
22         sumaTotal = sumaTotal + fibonacci(i);
23     }
24
25     printf("La suma de los primeros %d terminos de Fibonacci es: %d\n", N,
26           sumaTotal);
27
28     return 0;
29 }
```

Explicación: La función `fibonacci` se llama a sí misma reduciendo el problema en cada paso, hasta llegar a los casos base $n = 0$ y $n = 1$ que detienen la recursión. Desde `main`, un ciclo `for` recorre los índices de 0 a $N - 1$ acumulando cada término retornado.